

PAYMENT FAILURE ANALYSIS

Fintech Operations Analytics Project Report

Business Focus: Identifying concentrated payment failure patterns and translating them into actionable operational insights.

Tools: Excel / CSV → Python & Pandas → MySQL / PostgreSQL → Power BI

Project Type: End-to-End Data Analytics Case Study

Dataset Scope Note

This report treats the UPI Payment Failure Analysis dataset as a public practice/project source. The exact dataset filename, source URL, publication/version date, row count, and retrieval date should be recorded in the final project repository and replaced in the placeholders below.

Dataset version used: [Enter exact filename/version]

Source: [Enter source URL/project repository]

Retrieved on: [Enter date]

1. Executive Summary

Digital payments generate large transaction volumes, and even a relatively small failure rate can create significant operational impact. Payment failures can lead to customer frustration, repeat attempts, support contacts, merchant reconciliation issues, delayed settlement, and potential loss of transaction value. For a fintech operations team, the important question is not simply how many transactions failed, but where failures are concentrated, which segments experience disproportionately high failure rates, and where the business should investigate root causes first.

This project analyzes UPI payment transactions to identify failure patterns across payment methods, banks, devices, networks, transaction amounts, time of day, and bank-device combinations. The workflow follows a realistic analytics pipeline: data validation and cleaning in Python/Pandas, business-question analysis in SQL, and executive/diagnostic visualization in Power BI.

The final deliverable is designed to answer ten business questions and convert those findings into a dashboard that separates high-level operational KPIs from deeper diagnostic analysis. The analysis deliberately uses minimum-volume thresholds when comparing failure rates so that small segments with very few transactions do not dominate the interpretation.

2. Business Objective

The primary business objective is to help a fintech payment operations team locate concentrated payment failure patterns, quantify their operational significance, and identify segments that warrant further investigation.

A payment failure is an operational signal rather than merely a row-level outcome. If failures are concentrated in a particular payment method, bank, device category, network type, time period, or transaction-value segment, the pattern may indicate an integration issue, bank-side availability problem, network instability, device-specific behavior, transaction-risk controls, or another process-level constraint. The purpose of this project is to identify these patterns from transaction data before moving to root-cause investigation.

More specifically, the project aims to:

- Measure the overall scale of the problem: establish the total transaction volume, failed transaction count, overall failure rate, and failed transaction value.
- Locate concentration: determine which banks and payment methods account for the largest number of failed transactions.
- Identify disproportionately affected segments: compare failure rates by device, network, transaction amount, and hour of day.
- Control for sample size: apply documented minimum-volume thresholds before interpreting segment-level failure rates.
- Combine volume and rate: distinguish between a segment with many failures because it processes many transactions and a segment with an unusually high failure rate.
- Support operational prioritization: provide evidence that the payment team can use to decide which combinations or periods deserve deeper technical investigation.
- Translate analysis into an executive dashboard: create a KPI page for management and a diagnostic page for operational drill-down.

3. Business Context

In a fintech environment, payment reliability directly affects the customer and merchant experience. A failed payment may result in a customer retrying the transaction, switching payment methods, contacting support, or abandoning the transaction altogether. At scale, recurring failures can also

increase reconciliation effort and create operational workload.

The operations team therefore needs two complementary perspectives. The first is volume: where are the largest numbers of failed transactions occurring? The second is rate: where is failure unusually common relative to the number of transactions processed? A bank processing millions of transactions may have more failures in absolute terms but a lower failure rate than a smaller segment. Both measures are necessary for a fair diagnosis.

The project consequently treats failure analysis as a segmentation problem. Rather than stopping at an overall percentage, it breaks the dataset into operational dimensions and looks for concentration, abnormality, and interaction effects.

4. Analytical Scope

Dimension	Business purpose
Overall transactions	Measure baseline transaction activity and overall payment reliability.
Payment method	Identify methods with elevated failure rates.
Bank	Locate banks contributing the most failed transactions and compare bank-level failure rates.
Device type	Determine whether failure behavior differs across device categories.
Network type	Assess whether network conditions are associated with different failure rates.
Transaction amount	Identify amount ranges where failures are concentrated or unusually frequent.
Time of day	Identify hours with elevated failure rates.
Bank × device	Detect interaction patterns that may not be visible in separate bank/device summaries.
Failed transaction value	Measure the monetary impact of failures by bank and payment method.

5. Data Preparation & Baby-Step Data Cleaning

Data quality is critical because failure-rate calculations depend on correct transaction counts, valid statuses, reliable bank/device labels, and usable transaction amounts. The following sequence should be performed before the SQL analysis.

Step 1 — Check transaction_id uniqueness

Confirm that each transaction identifier is unique. Duplicate transaction IDs can inflate transaction counts and distort failure rates. If duplicates exist, investigate whether they represent genuine retries/events or duplicate records.

Step 2 — Convert timestamp and create date/month/hour

Convert the timestamp field to a proper datetime type. Create date, month, and hour fields to support daily, monthly, and hourly analysis.

Step 3 — Check missing status, bank, device and amount fields

Calculate missing counts and percentages for transaction status, bank, device type, and transaction amount. Decide whether records should be retained, corrected, or excluded based on the missingness pattern.

Step 4 — Standardize status and payment-mode labels

Trim whitespace, normalize capitalization, and map inconsistent labels to a common vocabulary. For example, SUCCESS, Success, and success should be treated consistently.

Step 5 — Check zero, negative and extreme amounts

Identify zero or negative amounts and inspect extreme transaction values. Do not automatically remove large values: determine whether they are legitimate before treating them as outliers.

Step 6 — Create failure_flag

Create a binary field where failed transactions equal 1 and successful transactions equal 0. Define the status mapping explicitly in the project documentation.

Step 7 — Create amount buckets

Create business-friendly transaction amount ranges so failure rates can be compared across value bands.

Step 8 — Document minimum-volume thresholds

Before comparing segment-level failure rates, record a minimum transaction-volume threshold. This prevents tiny groups from producing unstable percentages that could be misleading.

6. Suggested Python / Pandas Cleaning Workflow

```
import pandas as pd

df = pd.read_csv("your_dataset.csv")

# 1. Uniqueness
df["transaction_id"].is_unique
df["transaction_id"].duplicated().sum()

# 2. Timestamp
df["timestamp"] = pd.to_datetime(df["timestamp"], errors="coerce")
df["date"] = df["timestamp"].dt.date
df["month"] = df["timestamp"].dt.to_period("M").astype(str)
```

```

df["hour"] = df["timestamp"].dt.hour

# 3. Missing-value checks
cols = ["transaction_status", "sender_bank", "device_type", "amount"]
missing_summary = df[cols].isna().sum()

# 4. Standardize labels
df["transaction_status"] = df["transaction_status"].astype(str).str.strip().str.upper()

# 5. Amount checks
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")
df[df["amount"] <= 0]

# 6. Failure flag
df["failure_flag"] = (df["transaction_status"] == "FAILED").astype(int)

# 7. Amount buckets
bins = [-float("inf"), 500, 1000, 5000, 10000, float("inf")]
labels = ["<=500", "501-1000", "1001-5000", "5001-10000", ">10000"]
df["amount_bucket"] = pd.cut(df["amount"], bins=bins, labels=labels)

# 8. Volume checks
df["transaction_status"].value_counts(dropna=False)

```

7. Core Business Metrics

Overall Failure Rate = Failed Transactions ÷ Total Transactions × 100

Segment Failure Rate = Failed Transactions in Segment ÷ Total Transactions in Segment × 100

Failed Transaction Value = Sum of transaction amount for failed transactions

Share of Failed Value = Failed Transaction Value for Segment ÷ Total Failed Transaction Value × 100

These metrics should be reported together where appropriate. Failure rate describes frequency; failed transaction value describes financial exposure; failed transaction count describes operational workload.

8. SQL Business Questions & Analysis Framework

The following questions form the core SQL practice component. Each query should be saved in the project SQL file, followed by one concise business interpretation sentence.

Q1. What is the overall payment failure rate?

```

SELECT
COUNT(*) AS total_transactions,
SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END) AS failed_transactions,
ROUND(
100.0 * SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END)
/ NULLIF(COUNT(*), 0), 2
) AS failure_rate_pct
FROM payment_failure;

```

Business interpretation: Establish the baseline payment reliability of the complete dataset before analyzing individual segments.

Q2. Which payment methods have the highest failure rate?

```

SELECT
payment_method,
COUNT(*) AS total_transactions,
SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END) AS failed_transactions,
ROUND(
100.0 * SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END)
/ NULLIF(COUNT(*), 0), 2
) AS failure_rate_pct

```

```
FROM payment_failure
GROUP BY payment_method
ORDER BY failure_rate_pct DESC;
```

Business interpretation: Compare payment methods by both transaction volume and failure rate; avoid interpreting a very small method without checking its sample size.

Q3. Which banks process the most failed transactions?

```
SELECT
sender_bank,
SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END) AS failed_transactions
FROM payment_failure
GROUP BY sender_bank
ORDER BY failed_transactions DESC;
```

Business interpretation: Identify banks contributing the largest absolute number of failures, which can indicate where the largest operational workload is concentrated.

Q4. Which bank has the highest failure rate after applying a minimum transaction-volume threshold?

```
WITH bank_metrics AS (
SELECT
sender_bank,
COUNT(*) AS total_transactions,
SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END) AS failed_transactions
FROM payment_failure
GROUP BY sender_bank
)
SELECT
sender_bank,
total_transactions,
failed_transactions,
ROUND(100.0 * failed_transactions / NULLIF(total_transactions, 0), 2) AS failure_rate_pct
FROM bank_metrics
WHERE total_transactions >= 1000
ORDER BY failure_rate_pct DESC;
```

Business interpretation: Apply the documented minimum-volume threshold before comparing banks so that small-volume banks do not create unstable rankings.

Q5. Does failure rate vary by device type?

```
SELECT
device_type,
COUNT(*) AS total_transactions,
SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END) AS failed_transactions,
ROUND(
100.0 * SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END)
/ NULLIF(COUNT(*), 0), 2
) AS failure_rate_pct
FROM payment_failure
GROUP BY device_type
ORDER BY failure_rate_pct DESC;
```

Business interpretation: Determine whether device categories show materially different failure behavior and whether those differences persist at meaningful transaction volumes.

Q6. Does failure rate vary by network type?

```
SELECT
network_type,
COUNT(*) AS total_transactions,
SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END) AS failed_transactions,
ROUND(
100.0 * SUM(CASE WHEN transaction_status = 'FAILED' THEN 1 ELSE 0 END)
/ NULLIF(COUNT(*), 0), 2
) AS failure_rate_pct
FROM payment_failure
```

```
GROUP BY network_type
ORDER BY failure_rate_pct DESC;
```

Business interpretation: Compare failure behavior across network categories and consider whether one network segment deserves technical investigation.

Q7. Which transaction amount bucket has the highest failure rate?

```
SELECT
amount_bucket,
COUNT(*) AS total_transactions,
SUM(failure_flag) AS failed_transactions,
ROUND(100.0 * SUM(failure_flag) / NULLIF(COUNT(*), 0), 2) AS failure_rate_pct
FROM payment_failure
GROUP BY amount_bucket
ORDER BY failure_rate_pct DESC;
```

Business interpretation: Determine whether failure behavior changes with transaction value rather than assuming that all payment amounts have identical reliability.

Q8. Which hours of the day have unusually high failure rates?

```
SELECT
hour_of_day,
COUNT(*) AS total_transactions,
SUM(failure_flag) AS failed_transactions,
ROUND(100.0 * SUM(failure_flag) / NULLIF(COUNT(*), 0), 2) AS failure_rate_pct
FROM payment_failure
GROUP BY hour_of_day
ORDER BY failure_rate_pct DESC;
```

Business interpretation: Look for time-of-day concentration that may indicate peak-load, scheduled maintenance, network, or other operational patterns. Validate unusual hours against transaction volume.

Q9. Which bank × device combinations have the highest failure rates?

```
SELECT
sender_bank,
device_type,
COUNT(*) AS total_transactions,
SUM(failure_flag) AS failed_transactions,
ROUND(100.0 * SUM(failure_flag) / NULLIF(COUNT(*), 0), 2) AS failure_rate_pct
FROM payment_failure
GROUP BY sender_bank, device_type
HAVING COUNT(*) >= 500
ORDER BY failure_rate_pct DESC;
```

Business interpretation: Use an interaction view to identify combinations where the bank and device together show elevated failure behavior.

Q10. What share of failed transaction value comes from each bank or payment method?

```
SELECT
sender_bank,
SUM(CASE WHEN transaction_status = 'FAILED' THEN amount ELSE 0 END) AS failed_value,
ROUND(
100.0 * SUM(CASE WHEN transaction_status = 'FAILED' THEN amount ELSE 0 END)
/ NULLIF(
SUM(SUM(CASE WHEN transaction_status = 'FAILED' THEN amount ELSE 0 END))
OVER (), 0
), 2
) AS share_of_failed_value_pct
FROM payment_failure
GROUP BY sender_bank
ORDER BY share_of_failed_value_pct DESC;
```

Business interpretation: Quantify where the monetary impact of failed transactions is concentrated. Repeat the same analysis by payment method if required.

9. Minimum-Volume Threshold Strategy

Failure-rate comparisons can become misleading when a segment contains very few transactions. For example, a group with 2 failures out of 5 transactions has a 40% failure rate, but that percentage is much less stable than a group with 4,000 failures out of 10,000 transactions. The project should therefore document a threshold before making comparative statements.

A practical project convention is to use 1,000 transactions for bank-level comparisons and a lower or higher threshold for drill-downs depending on the dataset size. For bank × device combinations, the example SQL uses 500 transactions. These are analytical safeguards for this practice project, not universal industry standards. The final report should state the chosen thresholds and keep them consistent throughout the dashboard.

When a segment falls below the threshold, report its volume but avoid treating its failure rate as a stable comparative signal. This approach improves statistical discipline and makes the dashboard more defensible in interviews.

10. Power BI Dashboard Design

The recommended dashboard contains two pages: an Executive KPI Page followed by a Diagnostic Analysis Page. Every visual should map directly to one or more of the ten business questions.

Page 1 — Executive KPI Dashboard

Visual	Purpose	Related questions
Total Transactions KPI	Show transaction volume.	Q1
Failed Transactions KPI	Show absolute failure workload.	Q1, Q3
Overall Failure Rate KPI	Show baseline payment reliability.	Q1
Failed Transaction Value KPI	Show monetary exposure.	Q10
Failure Rate by Payment Method	Compare methods.	Q2
Failed Transactions by Bank	Show failure concentration.	Q3
Failure Rate by Bank	Show relative bank performance with threshold.	Q4

Recommended slicers: Month/Date, Payment Method, Bank, Device Type, Network Type, Transaction Status.

Page 2 — Diagnostic Dashboard

- Failure rate by device type — Q5
- Failure rate by network type — Q6
- Failure rate by transaction amount bucket — Q7
- Failure rate by hour of day — Q8
- Bank × device heatmap — Q9
- Failed transaction value share by bank/payment method — Q10
- Transaction volume should be displayed alongside failure rate wherever possible.

11. Recommended Power BI Measures

```
Total Transactions =  
COUNTROWS(payment_failure)
```

```
Failed Transactions =  
CALCULATE(  
COUNTROWS(payment_failure),  
payment_failure[transaction_status] = "FAILED")
```

```

)

Failure Rate =
DIVIDE([Failed Transactions], [Total Transactions], 0)

Failed Transaction Value =
CALCULATE(
SUM(payment_failure[amount]),
payment_failure[transaction_status] = "FAILED"
)

Failed Value Share =
DIVIDE(
[Failed Transaction Value],
CALCULATE(
[Failed Transaction Value],
ALL(payment_failure)
),
0
)

```

Use percentage formatting for Failure Rate and Failed Value Share. Use appropriate currency formatting for Failed Transaction Value if the dataset amount is in INR.

12. Visual Selection Guide

- KPI Card: overall transactions, failed transactions, failure rate, failed value.
- Clustered bar chart: payment method and bank comparisons.
- Column chart: hourly failure rate and amount-bucket analysis.
- Matrix / heatmap: bank × device failure rate.
- Donut or 100% stacked chart: use selectively for failed-value share where the number of categories is small.
- Slicers: use for date/month, bank, payment method, device, and network.

Avoid filling the executive page with too many visuals. The KPI page should communicate the scale and concentration of the issue quickly; the diagnostic page should support investigation.

13. End-to-End Project Workflow

Stage	Activity	Output
1. Data acquisition	Obtain the exact public practice dataset and record filename, version, source	Raw CSV / source documentation
2. Data profiling	Inspect columns, types, duplicates, missing values and distributions	Quality notes
3. Cleaning	Standardize labels, convert timestamps, validate amounts, create derived fields.	Cleaned dataset
4. Python analysis	Validate data and perform exploratory analysis.	EDA notebook / scripts
5. SQL analysis	Answer Q1-Q10 with reproducible queries.	SQL project file
6. Business interpretation	Write one business sentence under every SQL answer.	Insight log
7. Power BI	Build executive and diagnostic pages.	Interactive dashboard
8. Documentation	Document assumptions, thresholds, data limitations and findings.	Final project report
9. Portfolio packaging	Organize README, dataset note, Python, SQL and PBIX/reports into a project	Portfolio-ready project

14. Expected Business Insights

The analysis should ultimately answer questions such as: Is the problem broad-based or concentrated? Are failures driven primarily by a small number of banks or payment methods? Do particular devices or networks show elevated failure rates? Are failures concentrated at certain transaction amounts or hours? Are the highest failure-rate segments also responsible for a large share of failed transaction value?

Importantly, the project should not automatically interpret correlation as root cause. A high failure rate for a bank, device, network, or time period identifies a pattern for investigation; it does not by itself establish why the failures occurred. Root-cause analysis would require additional operational data such as error codes, API response codes, latency, bank availability, network telemetry, retry information, or incident logs.

15. Limitations & Assumptions

- The dataset is a public practice/project source and may not represent production UPI traffic.
- The exact dataset version must be recorded because public practice datasets can be modified or redistributed.
- Failure status is assumed to be correctly represented by the dataset's transaction-status field after standardization.
- A minimum-volume threshold is used to reduce unstable comparisons; the chosen threshold is a project convention.
- Bank-level failure rates should be interpreted as descriptive patterns within this dataset, not as a general assessment of real-world bank performance.
- The analysis identifies associations and concentrations; it does not prove operational causality.
- Extreme transaction amounts should be investigated before removal because legitimate high-value transactions may exist.

16. Final Deliverables

- 01_Data: raw dataset + cleaned dataset or cleaning script.
- 02_Python: data-cleaning and exploratory-analysis notebook/script.
- 03_SQL: SQL file containing Q1-Q10 and business interpretation comments.
- 04_PowerBI: PBIX file with executive and diagnostic pages.

- 05_Documentation: project README, dataset source/version note, assumptions and final findings.
- 06_Report: this detailed project report.

17. Portfolio / Interview Summary

This project demonstrates an end-to-end fintech operations analytics workflow. Starting with a public UPI payment dataset, the analysis validates transaction quality, standardizes operational fields, creates analytical features such as failure flags and transaction-amount buckets, and answers ten business questions using SQL. Python/Pandas is used for data preparation and validation, while Power BI converts the findings into an executive KPI dashboard and a diagnostic analysis page. The project emphasizes both transaction volume and failure rate, applies minimum-volume thresholds to improve comparison quality, and uses bank × device and failed-value analysis to move beyond simple totals toward operationally useful segmentation.

18. Project Completion Checklist

- Dataset filename/version documented.
- Transaction ID uniqueness checked.
- Timestamp converted to datetime.
- Date, month and hour created.
- Missing status, bank, device and amount fields checked.
- Status/payment-method labels standardized.
- Zero, negative and extreme amounts reviewed.
- Failure flag created.
- Amount buckets created.
- Minimum-volume threshold documented.
- Q1-Q10 SQL queries completed.
- One business sentence written for each query.
- Executive KPI dashboard created.
- Diagnostic dashboard created.
- Dashboard visuals mapped to Q1-Q10.
- Limitations and assumptions documented.
- Final findings added to README/report.